

循环工程：Anthropic 设计驱动智能体提示系统的手册

关于设计可自运行循环的实地研究

摘要

摘要—过去两年里，一串“XX 工程”术语一直紧跟模型发布节奏。本文考察其中最新的一个：循环工程。该术语于 2026 年 6 月由 Peter Steinberger、Boris Cherny 和 Addy Osmani 分别独立提出，并由 Osmani 首次以文字命名。不同于提示工程、上下文工程或运行框架工程，循环工程并不是教实践者把工作做得更好；而是让实践者彻底退出亲自做这项工作的岗位。我们定义该术语，将其置于运行框架之上的第四层，并把循环单次迭代拆解为五个动作——发现、移交、验证、持久化与调度——以及实现它们的六个组成部分。我们尤其关注生成器/评估器分离：经验上，让智能体给自己的输出打分时，它往往会自我表扬；而调优一个独立、持怀疑态度的评估器，远比让生成器对自身工作保持批判更可行。我们考察了三个已在实践中运行的循环，从一位工程师的晨间分诊，到 Stripe 企业级流水线每周合并 1,300 多个机器撰写的 pull request，并归纳出四种会悄然累积的成本——验证债、理解腐蚀、认知让渡和 Token 爆表。最后，我们给出构建第一个循环的具体配方。核心论点是：循环让生成几乎免费，而判断成为稀缺资源；同一个循环，由两个人构建，可能得到完全相反的结果。

关键词—Agentic AI，软件工程，自主智能体，编码智能体，生成器-评估器，调度，自动化。

I. 引言：循环工程到底是什么

提示工程课程还在售卖，上下文工程还墨迹未干，运行框架工程也才刚被写出来。如今循环工程也来了。过去一年里，这些“XX 工程”术语几乎伴随模型发布接连出现，让人忍不住翻白眼，也可以理解。但这次不一样。它不是关于把工作做得更好，而是把实践者完全从亲自做工作的岗位上移开。此前这些术语都默认有人坐在键盘前，逐行指挥智能体。循环工程删掉了这个前提。实践者不再处于循环之内，而是在循环之外构建循环。

A. 一句话定义

为这个术语命名并系统写下它的人，是 Google Chrome 团队工程师 Addy Osmani。他的定义很短：循环工程，就是不再由自己充当向智能体发提示的人，而是去设计替代自己的系统。不再逐行喂给智能体提示；而是去设计喂给它提示的系统。这句话的重心在于“替代你自己”。这是位置的转变——从做引擎的人，变成设计引擎的人。你写的不再是给智能体的话，而是一个会自动把话发给智能体的东西。

B. 一周之内，三个人点燃了引线

这个术语并非凭空诞生。2026 年 6 月的同一周内，几组人几乎同时撞上了同一件事。导火索是 Peter Steinberger（OpenClaw 作者）的一篇帖子，浏览量超过八百万：人们不该再直接向编码智能体发提示，而该去设计向它们发提示的循环。几乎同一时刻，Anthropic 的 Claude Code 负责人 Boris Cherny 也在表达同样观点——他不再直接提示 Claude，而是运行一些会提示 Claude 并判断该做什么的循环，而他的工作就是写循环。6 月 7 日，Osmani 在自己的博客中以《Loop Engineering》为题将此写出，纳入 Steinberger 和 Cherny 的说法，并于次日同步到 Substack。一次点火，一次回响，一个名字，都发生在一周之内。把这三种表述叠在一起，它们指向同一个动作：人们设计的对象，已经从智能体的单个行为，转向驱动智能体的整个系统。和此前的“vibe coding”一样，这个术语听起来或许粗糙，但它把一种工作方式变成了可以讨论的东西。

C. 为什么这个术语此时出现

值得追问的是：为什么三个彼此没有对过笔记的实践者，会在同一周用上同一个词。答案是，周边工具已经悄然跨过了一个阈值。编码智能体已足够可靠，能够在无人看管下完成非平凡任务；主要运行框架里已出现调度原语；而单次智能体运行的成本也已降到足够低，使得定时反复运行一个智能体不再显得浪费。当所有部件都已就位，把它们组合起来的动作就会同时对所有人变得显而易见。这个名字比实践晚了几个月：在有人称其为循环工程之前，人们就已经在写循环了；就像在生成器/评估器分离有名字之前，团队也早已在把写作者智能体和审阅者智能体配对使用。这种模式——实践在前，命名在后——值得记住，因为它告诉读者该去哪里寻找下一个术语。它不会来自一次模型发布。它会出现于这样一个时刻：某种新能力变得足够便宜，以至于此前不可想象的组合，开始成为日常。

D. 位于运行框架之上一层

这种转变可归结为两句话。在旧世界里，人坐在那里逐行向智能体发提示；它做完一件事就停下，等待。你是循环内部的人类时钟，而每一个 每一次推进都必须由人来触发。而在新世界里，人设计的是能自己推进的东西——它按定时器运行，生成助手来干活，并把自己的结果再喂回给自己。正如 Osmani 所说，循环工程位于运行框架之上一层：下层的运行框架为单次智能体运行备齐条件；上层的循环让它自己一遍又一遍地运行。这种变化也是身份的变化：从操作智能体的人，变成调度它的人。价值重心从“知道如何指挥”转向“知道如何构建循环，以及如何在循环里放一个能说不的检查”——而最后这部分最难，后文会解释。

II. 从提示到上下文再到循环

这些“XX 工程”术语并不是彼此替代；它们是层层堆叠的，每一层关注更大的对象。表 I 给出了这四层。每往上一层，关注单元就扩大一级：从一句话，到一个窗口，到一次运行，最后到一个自行运行的循环。图 1 将这四层画成一个栈，循环位于运行框架之上一层。

表 I 四层栈

层级	关注什么	核心问题
提示工程	写一个好提示	我该告诉模型什么？
上下文工程	当前窗口放什么	检索、总结、清除什么？
运行框架工程	武装一次运行	哪些工具、哪些动作、何为完成？
循环工程	在运行框架上调度	如何让它自行反复运行？

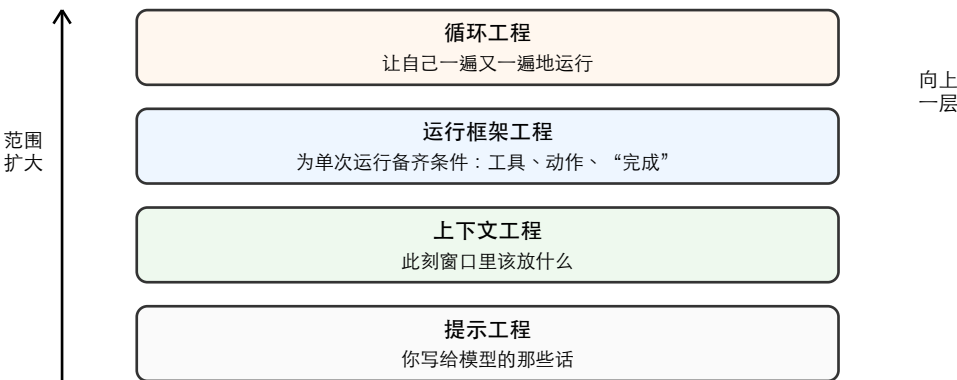


图 1. 四层栈。每一层关注的范围都比下一层更大；循环工程把运行框架留下的“等你来操作”自动化了。

A. 四个层级

提示是最底层，也是最广为人知的一层。它关注要告诉模型什么：措辞、示例、角色、语气。它的边界是一轮交互。问题在于，它默认每次都有人在场来递交提示。上下文把问题从“我该说什么”提升为“为了让模型解开这个问题，这个窗口里该放什么”。它关注模型的整个视野——该检索什么，如何总结，哪些过时信息该清掉。一个满是噪声的窗口，会浪费掉再好的提示。运行框架关注智能体完成单次运行需要携带什么：哪些工具、哪些动作、何时加载上下文、如何从失败中恢复、什么状态算完成。它为一次运行备齐条件；它并不让这次运行重复。循环把“等你来操作”自动化掉。前三层就位后，智能体可以干净地运行一次，但随后就停下。循环给它装上定时器，让它按计划唤醒，为并行工作生成子智能体，并把自己的输出作为下一轮输入再喂回去。

B. 向上一层到底增加了什么

有三个动词把运行框架和循环区分开来。按定时器运行：循环按计划唤醒，不需要按按钮。生成助手：转动起来的循环会拆分出子智能体——一个起草修改，另一个只负责在评审里挑毛病。自我回灌：循环产出的内容会成为下一轮自己的输入；昨天的发现写进文件，今天早晨它再读这个文件并继续做下去。这种跨对话的记忆，正是它成为循环而不是多次重复的一次性任务运行的原因。这种细粒度划分很重要，因为每一层的失效方式都不同，而那个能说“不”的检查，必须装在不同的位置。坏提示会当场暴露；坏上下文会表现为错误答案。但到了循环层，系统会在人睡觉时继续运行，改动从未看过的代码，并把自己的错误喂进下一轮——而错误可能几天都不被发现。层级越高，人离现场越远，错误堆积的时间越长。这也正是为什么循环工程真正的难点从来不是搭出循环，而是在里面放进某种能让它停下来的东西。

C. 各层失效的爆炸半径不同

考虑同一个底层 bug——智能体误读了某个函数的返回值——它在各层如何表现。在提示层，误读只会有一次交互中产生一个错误答案；人会立刻看到并重写提示。在上下文层，误读来自加载进窗口的过时文档；人会注意到答案错得很自信，于是清理上下文。在运行框架层，智能体只会基于误读行动一次——比如改了一个文件——但这次运行会结束，diff 清晰可见，而且在任何东西发布前都会有人审查。到了循环层，同样的误读会被写进状态文件，第二天早晨又作为既定事实读回来，并在多轮之中持续叠加。等到终于有人查看时，这个错误假设已经成了关键承重件。这是循环工程里最重要的直觉：一个错误的代价，会随着它在被发现前存活的轮次数量而放大；而循环从构造上说，正是一台把轮次数量最大化的机器。后文的一切——评估器、人工检查点、预算上限——都是为了缩短错误与其被发现之间的距离。

III. 一个循环的五个动作

“循环”这个词很容易被误解为空转。每一轮都会做具体的事：找出值得做的工作，把它交给智能体，验证结果是否正确，保存状态，然后决定下一步。少了五个动作中的任何一个——发现、移交、验证、持久化、调度——循环就无法转动，或只会原地打转。图2将这五个动作串成一轮，并送入下一轮。

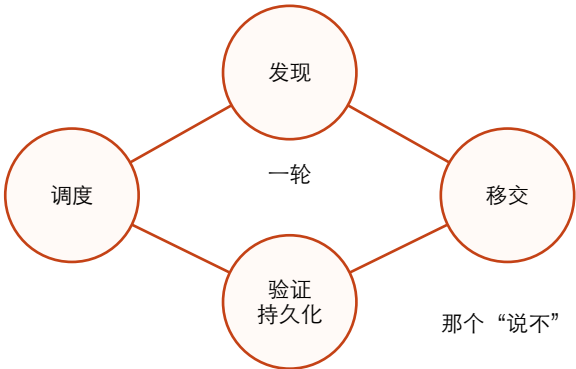


图 2. 一轮的五个动作。调度闭合循环，把未完成的一轮送入次日运行。验证是那个能够“说不”的动作。

A. 一个具体示例

Osmani 为自己搭了一个晨间分诊循环。早晨，自动化会自行启动。一个分诊 skill 读取昨天失败的 CI 测试、仍未关闭的 issue 以及最近的提交，并把结果写入 markdown 文件或 Linear 看板。对每个值得处理的发现，它都会打开一个隔离的 worktree；一个子智能体起草修复，第二个依据项目的 skills 和测试进行审查。连接器会自动创建 pull request 并更新工单。任何它无法处理的内容都会进入收件箱等待人工处理，同时状态文件会保留下来，便于第二天从今天停下的地方继续。整个过程无需人工介入，却会在该等人的地方准确停下。

B. 这些动作

发现决定这一轮该做什么。在这个例子中，分诊 skill 会读取 CI 失败、未关闭的 issue 和最近提交。关键在于让智能体自己找到工作，而不是别人给它一张清单。更重要的是，这个自动化触发的是 skill——被永久化的知识——而不是一大段贴进无人维护的 cron job 的说明。发现决定了整个循环质量的上限：如果浮现出来的工作毫无价值，那么其余四个动作做得再漂亮，也只是为无意义之事服务。移交把任务从调度系统转到实际执行工作的智能体手中。每个值得处理的发现都会得到自己隔离的 git worktree，这样多个智能体就能在不同目录中改代码而互不干扰。任务切分得越干净，后续验证和合并就越容易。验证是最容易偷工减料、也最不能省略的动作。第一个子智能体起草修复后，第二个子智能体会进行审查——使用不同的指令，有时甚至是不同的模型。写代码的智能体给自己的作业打分总会偏松；专门挑错的角色能抓住那些第一个智能体说服自己放过去的问题。这就是“那个能够说不的东西”。没有真正检查的循环，不过是智能体在对自己点头。持久化把结果落到对话之外仍能保留的地方：通过连接器提交 PR 并更新工单，为无法处理的事项建立收件箱，再用状态文件记录进度。循环的记忆不能只活在上下文窗口里；写进 markdown 或看板的内容不会遗忘。调度让一轮变成循环。分诊每天早晨自动运行，而状态文件让未完成的发现能够带到第二天，第二天也会自行接续。正如 Osmani 所说，自动化才让循环成为真正的循环，而不只是你做过一次的一次运行。表 II 总结了这五个动作。

表 II 五个动作与分诊循环的对应关系

动作	作用	分诊循环中
发现	自行找出本轮工作	skill 读取 CI / issues / commits
移交	隔离地交给智能体	每个发现打开一个 worktree
验证	换另一个智能体说“不”	第二个子智能体对照测试审查
持久化	把状态写到对话外	PR + inbox + 状态文件
调度	让它一轮轮转起来	早晨自动化自行运行

IV. 六个部件：循环由什么构成

如果说动作描述的是一轮中发生了什么，那么部件描述的就是要让它真正转起来必须具备什么。两者是一一对应的：发现依赖 skills，移交依赖 worktrees，验证依赖子智能体，持久化依赖记忆，调度依赖自动化。自动化让循环能够自行启动，挂接在某个日程或触发器上。没有调度，手里只有一次运行，而不是循环。自动化应触发一个具名 skill，而不是 cron job 里一大段说明。调度不止一种形式——本地式（机器必须开着）和云端式（机器关机也能继续轮转）。Worktrees 是 git 内置机制，用于在一个仓库中维护多个独立工作目录。它的价值会随着并行度提升而放大：两个智能体同时写同一个文件，和两个工程师同时提交同几行代码，是同一种麻烦。Worktrees 把并行从“能跑但很乱”变成“能跑且整洁”。Skills 把项目知识永久化到单个文件（SKILL.md）中，这样智能体无需每轮都重新推导上下文。Osmani 把它们抵消的成本称为 intent debt：反复解释“这个项目是什么、规则是什么、陷阱在哪里”的代价。skill 可以复用，也可以维护；一大段 prompt 不行。连接器（基于 MCP，即 Model Context Protocol）把循环接到外部世界上：问题跟踪器、数据库、staging API、Slack。一个只能看到文件系统的循环，是个很小的循环。连接器决定循环的视野半径，而为一种工具编写的连接器，往往也能直接接到另一种工具上使用。子智能体把负责写的那个和负责评判的那个分开。当一个智能体既是选手又是裁判时，裁判就会偏袒自己。反直觉的是，把一个独立裁判调得挑剔，远比让生成器对自己的作品苛刻容易得多；这也是为什么循环会保留一个额外智能体，而不是让同一个智能体审计自己。记忆是存在于单次对话之外的持久状态——一个 markdown 文件或一块看板。上下文窗口一清空，智能体就什么都不记得了；要让循环今天从昨天停下的地方继续，记忆就必须落到磁盘上。智能体会忘，repo 不会。记忆不是上下文：上下文是智能体这一轮看到的内容，刷新时会被清空；记忆会跨轮次、跨天保留。表 III 将这六个部分映射到五个动作。六个部分都到位后，循环就有了骨架：自动化让它动起来，工作树避免它自相冲突，技能让它不必重复劳动，连接器让它看见外部，子智能体让它纠正自己，记忆让它记住事情。但把它搭出来只是开始：同样一组部件，由两个人构建，结果可能完全相反。

表 III 六个部分映射到五个动作

部分	含义	对应动作
自动化	按计划/触发器运行	调度
工作树	并行智能体的隔离目录	移交
技能	持久知识；偿还意图债	发现
连接器	通过 MCP 接外部系统	持久化/发现
子智能体	生成器与裁判分离	验证
记忆	磁盘上的持久状态	持久化

V. 生成器与评估器

一个循环里最难的部分，不是让智能体跑起来，而是往里面放进一个能说“不能过”的东西——而写代码的那个智能体，恰恰最不可能这么说。

A. 它总是在夸自己

让一个智能体给自己刚产出的东西打分时，它往往会自信地夸赞它，即使人类一眼就能看出质量平平；Anthropic 工程师 Prithvi Rajasekaran 在构建长时运行应用时观察到了这一点。这不是聪明不聪明的问题；这是在给自己的作业打分。上下文 代码写出时所处的上下文里，已经塞满了“为什么要这样写”的理由，所以当智能体回头看自己的输出时，它看到的不是结果，而是导向这个结果的一整条自我说服链。在循环里，这个缺陷会被放大：如果每一次“这样够好吗”都由刚刚写下它的智能体来裁定，它每一轮都会对自己点头；而且跑得越久，就越偏离真实质量。

B. 调一个怀疑者，而不是修一个谦虚作者

让生成器变得更会自我批评，效果通常不好。Rajasekaran 发现，把一个独立评估器调成怀疑型，远比让生成器批评自己的作品更可行。差异在于结构，而不是措辞：你无法要求作者跳出自己的视角，但你可以换上另一个有着完全不同指令的智能体，让它从零开始看代码，不带任何自我说服。这个思路借自生成对抗网络（GANs）——一个网络负责生成，一个网络负责挑错——移植到“生成器负责写、评估器负责审”的结构中。图 3 展示了由此形成的循环。

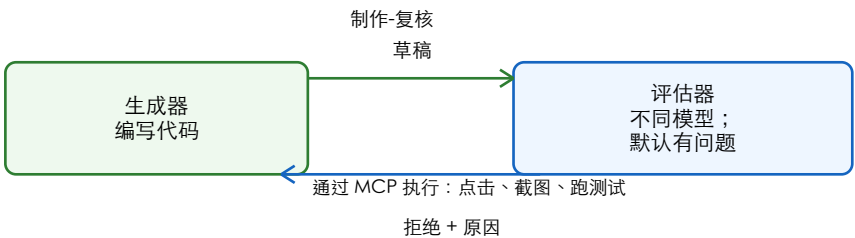


图 3. 生成器与评估器作为独立智能体。评估器不带生成器的自我说服，默认持怀疑态度，并通过执行而非只读来判断行为。

C. 评估器应当行动，而不只是阅读

只替换智能体还不够。如果评估器只读代码，它判断的是“看起来对不对”，而不是“跑起来对不对”。在前端任务中，Rajasekaran 把评估器接到 Playwright MCP 上，让它像 QA 工程师一样打开页面、点击按钮、截图并检查 DOM。这样，判断依据就从“这段 JSX 看起来没问题”转成了“我点了按钮，页面发生了跳转，这是截图”。替换底层模型也有帮助：同一个模型即使换了新指令，往往仍会保留自己的盲点。社区里一个常见的校准方式，是要求评估器默认假设代码有问题，除非被证明没问题——默认姿态应当是怀疑，而不是信任。

D. 在产品中：把 /goal 用在停止条件上

Claude Code 用 /goal 把这种结构做成了一个原语：给智能体一个条件，让它持续运行，直到条件满足。一个有代表性的评估器配置和停止条件如下所示。

```
# 评估器智能体 ( .claude/agents/reviewer.md )
ROLE: 对抗式代码审查者。
ASSUME: 证明无误前，都视为有问题。
DO NOT praise. Find what fails.

CHECK, in order:
1. 能跑吗？（执行，别只读）
2. 测试：运行，并贴真实输出。
3. 作者漏掉的边界情况。
4. 行为是否符合工单？

USE Playwright MCP: 打开页面、点击、截图、检查 DOM。按行为评判，不按意图。
VERDICT: 所有检查通过才 PASS；否则 REJECT，并列原因。

# 停止条件，由一个全新小模型判断
/goal test/auth 中所有测试通过，且 lint 步骤干净
```

关键在于，每一轮之后都有一个小而快的模型检查该条件是否成立；若不成立，就再跑一轮，而不是交还控制权。是否完成由一个全新模型决定，而不是执行工作的那个模型。这就是制作-复核原则——银行业已沿用数十年，大额转账的录入人与审核人必须不同——应用到停止条件上。（Codex 通过自动化加智能体配置达到同样能力；不应把 /goal 和 /loop 混淆，后者只是按间隔重跑。）一个循环的下限由其评估器决定。生成器的水平决定循环能产出什么；评估器的水平决定它不会产出什么。在结构上分离生成与判断，把评估器调成怀疑者，让它通过实际操作来验证，并把最终裁决交给一个全新模型——这四步，才让循环真正学会说“不”

”。

VI. 循环出错的五种方式

在转向有效的循环之前，值得先梳理它们失败的方式，因为失败比成功更有启发，也更常见。下面每种反模式都对应五个动作中的某一步被跳过或执行不当。



图 4. 每种反模式都对应一个被跳过的动作。五种失败与单轮中的五个动作一一对应。

A. 点头循环（跳过验证）

这是最常见的失败。循环在跑，智能体在写代码，又由同一个智能体宣布它没问题。没有独立检查时，每一轮都会产出自我批准的结果，循环便以机器速度累积看似合理的错误。症状是：几百轮下来，循环从未对自己说过一次“不”——对任何真实工作负载而言，这在统计上都不可能，因此足以证明根本没有真正检查。修复方法是上一节的生成器/评估器拆分。

B. 失忆循环（跳过持久化）

循环发现了好工作，也做完了，然后却忘了这件事发生过，因为结果只存在于一个已被清空的上下文窗口。下一轮会再次发现同样的工作，或者更糟，重做一遍并与第一次尝试冲突。症状是循环没有累计进展：每天早上都从同一个地方开始。修复方法是把状态文件落盘——智能体会忘，代码库不会。

C. 手动循环（跳过调度）

一个有四个好动作却没有自动化的循环，并不是循环；它只是一个由人手动运行、然后忘了再运行的脚本。它在建成当天看起来很惊艳，在注意力转移那天就悄然停止。症状是：循环上次运行，就是演示那天。修复方法是真正的触发器——定时器或事件——不能依赖人去记住。

D. 盲目循环（跳过发现）

人类每天早上仍要把工作交给循环——“修这三个 bug”——所以循环只自动化了执行，没有自动化发现。这节省的比看上去少，因为决定做什么往往才是成本最高的部分。症状是人类早上仍在花时间决定循环该做什么。修复方法是把发现训练成一种技能，让循环自己浮现工作。

E. 缠结循环（跳过移交）

循环并行运行多个智能体，却让它们都改同一个工作目录，于是修改彼此冲突，合并乱成一团，没人能理清。这种症状只在并行时出现：单智能体循环看起来没问题，而问题会在五个智能体首次同时运行的早晨暴露。修复方法是每个任务一个隔离的工作树。图 4 将五种反模式映射到它们违反的动作。这五种问题并非彼此独立。缺少验证的循环往往也缺少持久化，因为一个团队对某种检查粗心，通常对其他检查也会粗心。实践中它们会成簇出现：纪律严明的循环会装上五个动作；仓促的循环往往只装发现和移交——这两者会产出可见结果——而跳过会带来安全性的另外三项。下一节展示三个装齐了五个动作的循环。

VII. 你睡觉时仍在运行的循环：三个真实案例

三个公开案例规模差异极大，却共享同一副骨架：触发器负责启动，一组约束让它不脱轨，末端还有一个人工检查点。“在你睡觉时运行”从来不是模型有多强，而是这副骨架有多稳固。

A. 一位工程师的早晨

Osmani 的分诊循环（第 III 节已拆解）每天早晨自动运行。有一个细节值得再次强调：自动化调用的是技能，而不是一大块指令 被粘进一个谁也不会再更新的日程。这就是一个人能跑的循环——一个人，一台机器，每天早上把杂活做完。

B. Stripe 的 Minions：每周 1,300 个 PR

在企业级规模上，值得研究的案例是 Stripe 的 Minions：据 Stripe 工程师 Steve Kaliski 在 How I AI 播客中的描述，每周合并超过 1,300 个 pull request，且没有一行是手写的。触发方式很轻——在 Slack 里 @ 机器人，或加一个emoji 反应。真正让它可靠的，是模型启动前那段流程：先由确定性编排器组装上下文，扫描链接、拉取 Jira、查找文档，并用 Sourcegraph 加 MCP 定位相关代码。让 LLM 自己找上下文，是最不可控的环节，因此这部分——凡是规则能硬编码的——都从模型手里拿走。凡是确定性逻辑能解决的，就绝不交给概率模型；这条线画在哪，决定了循环是否可靠。最违反直觉的一点是：Minions 并不是靠更强的模型。它基于开源工具 Goose 的一个分支，核心主张是：可靠性来自约束质量，而不是模型大小。其架构把确定性关卡与创造性的 LLM 步骤交错编排，如图 5 所示——智能体写代码，硬编码流水线运行 linter 且智能体无法跳过，智能体修复 lint，再由硬编码步骤执行 commit。沙箱是运行在 EC2 上的 Devbox，按“cattle not pets”的方式运维：每个环境都可随时替换，因此一千多个智能体可同时运行而互不干扰。值得注意的是，这 1,300 个 PR 仍由人工审查——人并未离场，只是从编写者换到了审查者的工位。

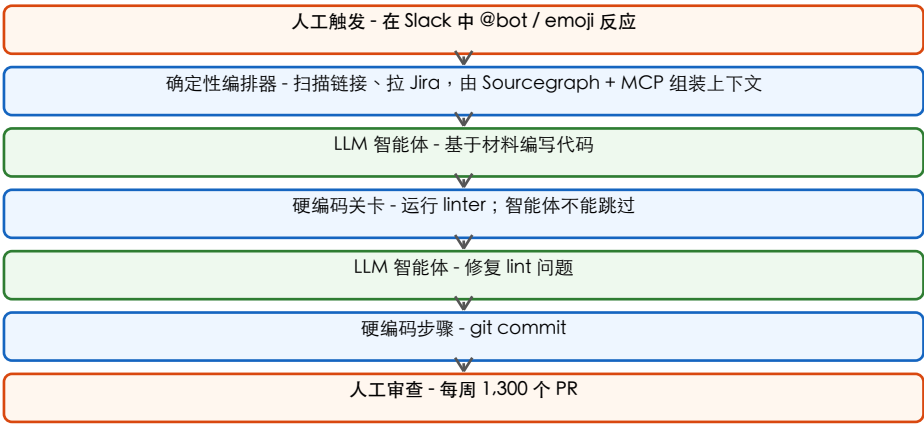


图 5. Stripe 的 Minions 流水线。确定性关卡（蓝）与 LLM 步骤（绿）相互咬合；凡是规则可约束的，都留在概率模型之外。可靠性来自约束，而非模型规模。

C. “你睡觉时运行” 实际依赖什么

本地 /loop 和桌面计划任务都要求机器开机；一关机，循环就停。若想在关机时继续运行，正确答案是 Cloud Routines 或 GitHub Actions 的 schedule 触发器。表 IV 对比了这些选项。若想高频运行且能看到本地文件，用本地 /loop，代价是机器必须保持

表 IV 调度选项对比

	Cloud	Desktop	/loop
运行位置	云端	机器	机器
机器开机？	否	是	是
会话打开？	否	否	是
最小间隔	1 小时	1 分钟	1 分钟
可见本地文件？	否	是	是

开机。若想摆脱本地状态独立运行，就上云端，代价是最短间隔一小时，且每次都要从干净克隆开始。没有一种调度器能包打天下。对广泛流传的数据还需谨慎：诸如“Claude Code 大约90% 由自己编写”或大型迁移提速之类说法，多半是二手概述，应只视作粗略参考。本文这三个案例都可追溯到一手来源，比单个听起来很惊人的数字更站得住脚。

D. 在实践中选择调度器

本地调度还是云端调度，不是偏好问题；它机械地取决于一个问题：循环的工作是否绑在本地机器上，还是可以离开？两个具体场景足以说明。假设某个循环必须每分钟检查一次本地开发服务器——那它只能本地运行，因为云端看不到你笔记本上的进程，而且云端间隔也不能低于一小时。再反过来：假设某个循环应在凌晨三点扫描仓库中的 open issue，并在合适时创建 pull request——这类工作就绝不该绑在笔记本上，因为笔记本会合盖、会断电，也会被人带走。对后一种情况，正确答案是云端调度或 CI 的 schedule 触发器，运行在一台人睡觉时仍保持唤醒的机器上。需要避免的误区，是把本地重跑当成“睡觉时运行”的全部。本地重跑的意思是“我在场时多跑几轮”；云端调度的意思是“我不在时也继续跑”。这是两种不同能力，把它们混为一谈，就会让人一合上电脑盖，原以为自治的循环悄然停下时大失所望。更诚实的表述是：本地调度换来高频率和本地文件访问，但代价是机器必须持续开机；云端调度换来真正自治，但代价是间隔更粗、且每次运行都是全新克隆。没有单一调度器能包办一切，成熟的循环往往两者并用——本地负责紧密的内层检查，云端负责夜间扫荡。

E. 同一种能力，两套工具链

本文中的命令来自 Claude Code，但这些能力并不专属于它。Codex 以不同名称提供同样的五个器官，而且为一边编写的连接器，往往可以直接迁到另一边使用。表 V 将它们并列列出，免得读者把命令名认错门。要点在于，循环工程是一组能力，而不是某个产品：调度、运行直到满足条件、并行隔离，

表 V 两条工具链上的同类能力

能力	Claude Code	Codex
调度	/loop worker	Automations tab
满足前持续运行	/goal	automation rerun + judge
并行隔离	--worktree	background worktree
子智能体	.claude/agents/	.codex/agents/
外部连接	MCP + plugins	MCP connector
显式技能	SKILL.md	\$skill-name
离机运行	Cloud Routines	cloud (planned)

子智能体、外部连接，以及显式技能调用。无论团队使用哪条工具链，要问的是这六项是否齐备，而不是由哪个品牌的命令提供。

VIII. 代价：四个不会自行清空的标签页

一个会自行运行的循环，同时也是一个会自行犯错的循环。它运行得越欢快，出错就越安静。四种代价会逐步累积，而在循环运行时，没有一种会发出警报。验证债。每个被打开并合并的 PR 都节省时间，但省下的时间会变成尚未验证、等待偿还的输出。问题藏在测试未覆盖之处，藏在“能跑”和“正确”之间的缝隙里，持续累积，直到某个发布的清晨一次性爆炸。防线是独立的评估器——与执行工作的智能体不同的另一个智能体。理解腐蚀。循环交付你未亲手写下的代码越快，现有内容与你实际理解之间的差距就越大。读代码比写代码更乏味，而循环已经把写代码接管了；代码库持续增长，你脑中的地图却停滞不前。它不会报警，直到某个 bug 钻进一个从未读过的角落。防线是定期阅读循环的输出，并强迫自己解释几处改动；一旦解释不清，就说明地图需要更新。认知让渡。当循环自行运行时，人很容易不再持有判断，只是照单全收。这是前两者在态度层面的版本：不是“没时间”，而是“懒得管了”。循环越可靠，就越容易把判断外包出去。防线只有一句话——循环可以执行，但不能决策。人至少必须保有说出“这不对”的能力。Token 爆表。这是唯一会直接打到账单上的代价，而且事先很难估计：循环会孵化辅助者、反复重试、一轮又一轮运行，所以一个 bug 就可能整夜空转，最后产出一张陌生的账单，而不是修好的代码。防线是在上线前设定硬上限——单次预算、每日预算、最大重试次数——这样空转的 bug 就不会烧掉整晚配额。这四者有一个共同特征：循环运行时一片沉默。循环工程最迷人的地方，在于它让一个人做出一个团队的工作；最危险的地方也恰恰在这里，因为团队会彼此争论，而一个人加上一堆循环，很容易变成无人争论的回音室。

A. 复利式债务的一个算例

设想一个循环在一夜之间开出二十个 pull request，而且测试全绿。表面看这是一场胜利。但假设其中有三个包含测试未覆盖的隐蔽错误。在没有独立评估器的情况下，这三个会被合并——这就是验证债。因为人类合并了二十个 PR 却没有阅读，它对代码库的心智模型如今落后了二十处改动——这就是理解腐蚀。因为循环运行得如此顺滑，这个人第二天早上甚至不再阅读下一批结果——这就是认知让渡。而因为循环整夜自由地产生辅助者并不断重试，账单变成预估的三倍——这就是 Token 爆表。这三个被埋下的错误，如今坐落在一个人类已不再完全理解的代码库中，由一个已经停止查看的人把守，最终只会在其中某个浮现为生产事故时才发现。这个算例要说明的是，这四种代价不是一组彼此独立的风险；它们是同一个失败的四张面孔。它们彼此强化：循环产生的未经验证输出越多，人类理解得就越少；理解越少，让渡就越多；让渡越多，循环就在无人看守下运行得越久，账单也越大。图 6 展示了这种强化循环。防御这四者的方法其实相同：让一个人始终保有说“不”的能力，并装上一道不必等人醒着才能运行的检查。

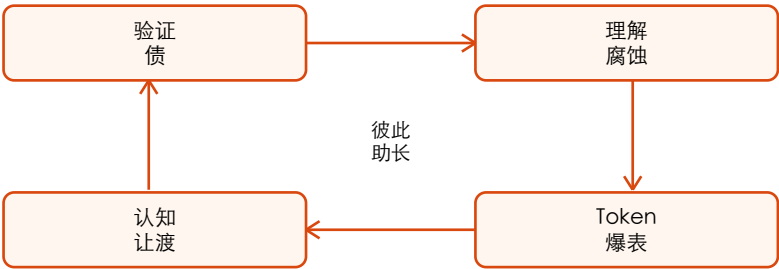


图 6. 四种代价彼此强化。未验证的输出会侵蚀理解，进而诱发让渡，让循环运行更久、花费更多，又产生更多未经验证的输出。

IX. 继续做工程师，而不只是那个按下开始的人

同一个循环，由两个人构建，最终可能走向完全相反的结果——差别并不在循环本身。正如 Osmani 所写，两个人可以构建同一个循环，却得到相反结果。一个人用循环在自己已掌握的事情上更快前进：他们读代码，牢牢把握方向，而循环只是放大他们原本已有的判断。另一个人则用同样的循环，让自己再也不必理解任何东西。六个月后，一个变得更强大，另一个则成了某台自己看不懂机器的看门人。循环不是一种其质量由工具本身固定决定的工具。它强大到会原样放大一个人带入其中的一切：带入理解，它就放大大理解；带入懒惰，它就放大大懒惰。它是一个忠实的乘号，而它所乘上的，是人本身。循环让生成变得极其廉价——代码、计划、PR、修复，几乎都变得免费。真正稀缺的是判断力：知道哪个方案是对的，哪条线该停，哪个输出虽然能跑却从根上就是错的。循环可以生成一百个选项，却无法真正做出选择；或者说，它依据的是“看起来合理”，而不是“实际上正确”，而这两者之间的差距正是工程师存在的理由。因此，循环工程并不会贬低判断力；它只是剥离掉所有不需要判断的部分，让判断成为最后剩下的一切。循环会执行赋予它的逻辑，但并不理解人为什么要构建它、真正想要什么，或哪些环节宁愿亲自盯着。这些边界——哪里设人工检查点、哪里让它自行运行——无法从循环本身读出；它们只存在于构建者脑中，必须一项项写进去。你以什么心态设计，循环就会长成什么形状。带着“尽快把自己解放出来”和“我仍然要做工程师”两种心态构建的两个循环，代码上可能有九成相同；差别只在一两个检查点，而正是这些差别决定了六个月后，你是站在循环之上，还是被它掏空。Osmani 的结尾最该记住：去构建循环，但要以一个打算继续做工程师的人去构建，而不只是那个按下开始按钮的人。

X. 判断力的经济学

上一节提出了一个值得单独审视的观点：循环让生成变得廉价，却让判断依然稀缺。这不是口号，而是一条经济学观察，并会影响团队应如何组织。

A. 什么会变得充裕

当一种资源变得充裕，它的价格就会下降，围绕它组织的活动也会随之重组。循环让代码、方案、修复和 pull request 变得充裕——一个拥有良好循环的工程师，可以产出一个小团队的工作量。那些过去占据工程师一天时间的工作，打字、样板代码、机械式重构，成本都趋近于零。一个自然的第一反应是，这会让工程师贬值。事实恰恰相反，但前提是工程师抓住了那种稀缺资源。

B. 什么仍然稀缺

稀缺资源，是决定保留哪些充裕产出的判断力。循环可以生成一百个候选实现；它无法告诉你哪个是对的，只能告诉你哪个看起来合理，而“看起来合理”与“实际上正确”之间的差距，恰恰就是工程存在之处。随着生成趋近免费，工程师的全部价值都会集中到这道缝隙里。剩下的工作，就是纯粹的判断，不再被过去包围它的机械劳动所稀释。这带来一个不太舒服的含义。若一名工程师的价值主要来自机械劳动——打字快、广泛记忆 API、愿意埋头堆样板代码——那么这种价值会蒸发，因为循环会免费完成这一切。若一名工程师的价值在于判断力，这种价值就会被放大，因为循环会把他们的好决策放大执行一百倍。同一种工具会拉大这两类工程师之间的差距。它不会平等抬升每个人；它只会放大每个人原本带来的东西。

C. 放大器是双刃剑

因为循环是判断力的放大器，判断失误也会被放大。在旧世界里，一个错误决策的代价，只是一段手写出来的错误代码，影响范围有限，而且推进得足够慢，来得及发现。到了新世界，一个错误决策会被机器忠实地、大批量地执行上一百次，而机器不会停下来问它是否正确。循环拿掉了过去常常把工程师救回来的慢挡。你不能再指望流程慢到足以在中途发现错误，因为流程已经没有慢挡了。这抬高了循环唯一做不到之事的赌注，也正因此，坚持做工程师不是可有可无的情怀，而是运行上的必要条件。

XI. 运行纪律

如果判断力是稀缺资源，实际问题就是如何把它花得值得。结合上文的案例与代价，有三条纪律值得作为常规实践明确下来。

A. 一定要读样本

对抗理解腐蚀的办法，不是把循环产出的所有东西都读一遍——那样就失去了意义——而是每天读一个有代表性的样本，并强迫自己解释每个抽样变更：它做了什么，为什么这样做。解释不出某个变更，就是一个精确的信号，说明你的心智地图已经落后于代码库；在一个安静早晨通过抽样 PR 发现这一点，远比在糟糕的一天通过生产事故发现，要便宜得多。样本不必很大；关键是要规律，而且要真的认真检查。

B. 先设上限，再发版

对抗 Token 爆表的办法，是在循环第一次无人值守运行之前就设好硬上限，而不是等第一张意外账单来了再补。单次预算、每日预算和最大重试次数，三者结合，才能确保某个 bug 在夜里空转时，不会吃掉整份配额。这些数字的首要目的并不是省钱；它们是断路器，把开放式风险变成有边界的风险。没有上限的循环，等于把花钱权限委托给了它自己的 bug。

C. 始终留一扇门开着

对抗认知让渡，靠的是结构，而不只是态度。至少在循环里嵌入一个会为人暂停的检查点，不是因为人总会介入，而是因为这个暂停的存在，能让人始终处在“可以介入”的位置上。把每一扇门都焊死、赌自己永远不需要进去的工程师，会在真正必须进去的那天发现，钥匙已经不在自己手里。留着一扇门开着的工程师，则随时都能走进去，看看循环在做什么。这两个循环的差别，只在于一个检查点 代码；六个月后究竟由谁掌控，差异却极大。

XII. 今天就搭建你的第一个循环

Stripe 的流水线是终点，不是起点。第一个循环应当小到几乎不像系统——只是一个按定时器检查某件事的小东西。第一步：运行 `/loop`。Claude Code v2.1.72 后提供，它会按间隔重复执行同一任务。它受会话作用域限制，周期任务七天后过期，并且运行在本地机器上；机器关机就会停止。

```
/loop 5m check the deploy # 固定每 5 分钟
/loop check the deploy # 智能体自行掌握节奏
/loop # 运行 .claude/loop.md
```

第二步：读取 CI 和 issue；先做分诊。重复跑一行命令不算循环。给它一个提示，让它每天早上查看三样东西，并列值得处理的项。调度加自动发现，是循环入门级。发现逻辑应放在 skill 里，而不是调度里。

```
# .claude/skills/morning-triage/SKILL.md
NAME: morning-triage
WHEN: 每天早晨由自动化调用。

READ:
- 昨天以来失败的 CI
- 过去 24h 新开的 issue
- 上次运行后的 commit

JUDGE: 每项是否值得现在处理？跳过噪声，只保留可执行发现。
OUTPUT: 写入 ./state/triage.md
```

第三步：加入状态文件。不要把结果留在聊天窗口里。把每个发现以及处理进度写入 markdown 文件（或 Linear 看板）。智能体会遗忘；仓库不会。

```
# ./state/triage.md ( 循环的记忆 )
| finding | source | status |
| auth test flaky | CI #4821 | fixing |
| null deref | issue 92 | PR open |
| stale dep | commit a3 | inbox |
```

第四步：加入评估器。这是最关键也最容易跳过的一步。Claude Code 的 `/goal` (v2.1.139 后) 会持续运行，直到满足某个条件，并由不同模型判断该条件是否成立。

```
/goal test/auth 中所有测试通过，且 lint 步骤干净
```

第五步：加入工作树以实现并行。使用 `--worktree` (或 `-w`) 为每个后台智能体打开独立工作树，避免彼此踩踏。

```
# 每个发现一个隔离 worktree
claude --worktree fix/auth-test "draft the fix"
claude --worktree fix/null-deref "draft the fix"
```

表 VI 的六个要素中，前两个决定循环能否运行；后四个则决定它一旦运行后是否会惹出麻烦。新手最常见的情况是只做了前两个，结果就是一个无人看管、无人能停、只会自我点头的循环。最后的建议很简单：第一个循环可以小，但那个会说“否”的检查点和人工审查点必须完整装上。

A. 一个完整的首个循环，附注释

为了让检查清单更具体，下面给出一个最小但完整的循环，装齐了全部六个要素。它足够小，可以一口气读完，同时包含真实循环所需的每个器官，只是按比例缩小了。

```
# 1. 调度 -- 真实触发器
# (.github/workflows/triage.yml)
on:
  schedule:
    - cron: '0 6 * * *' # 每天 06:00 云端

# 2. 发现 -- skill，不是一堵提示词墙
run: claude --skill morning-triage

# 3. 持久化 -- 磁盘状态
# skill 写入 ./state/triage.md 并 commit

# 4. 移交 -- 每个发现一个 worktree
for finding in $(parse ./state/triage.md); do
  claude --worktree "fix/$finding" \
    --goal "tests pass and lint is clean" \
    "draft a fix for $finding"
done

# 5. 验证 -- 新模型裁决
# /goal 每轮后检查；第二审查者挑错

# 6. 人工审查 -- 敞开的门
# 只开 PR，不自动合并；不确定进 ./inbox/
```

自上而下阅读，这六条编号注释就是检查清单的六个要素，每个都以两三行实现。cron 行是调度；skill 调用是发现；被 commit 的状态文件是持久化；按发现拆分的工作树是移交；`/goal` 停止检查加审查者是验证；而绝不自动合并加 inbox 规则则是人工审查这一点。具备全部六个部分的循环，哪怕再小，也是真正的循环。缺少其中任何一个的循环，都只是第 VI 节五种失败中的一种换了伪装。

B. 安全地扩展循环

一旦最小循环跑通，诱惑就会出现——更多发现、更多并行智能体、更短间隔。安全的扩展顺序是最后才加并行，而且要在各项检查已被验证之后。先提升循环能发现什么，再提升并行执行多少；先证明评估器能抓住真实错误，再信任它一次性把关许多智能体。Stripe 的案例是这条路径的终点，不是起点：它的可靠性来自多年对确定性关卡的加固，而不是一开始就做大。循环必须先证明自己能拦住一个坏智能体，才有资格运行更多智能体。其余内容不在本札记里；在终端里。

XIII. 附录 A：带注释的分诊技能

发现这一动作依赖的是技能，而不是一大段说明，因为技能可以复用和维护，而粘贴的提示会在无人更新的调度中逐渐腐蚀。下面给出文中多次提到的晨间分诊技能的更完整版本，并加注释说明各部分如何服务于各个动作。

```
# .claude/skills/morning-triage/SKILL.md
---
name: morning-triage
trigger: 每日自动化调用
---

## 读取 ( DISCOVERY 输入 )
- 自上次运行以来失败的 CI
- 过去 24 小时新开的 issue
- 昨天以来合并的提交
- 上一次 ./state/triage.md

## 判断 ( 设定上限 )
对每个候选项判断：
- 现在可执行，还是噪声？
- 是否阻塞发布？ -> priority
- 是否已被跟踪？ -> skip
只保留今天值得开工作树的项。

## 写入 ( PERSISTENCE 输出 )
追加到 ./state/triage.md:
| finding | source | priority | status |
提交该文件，便于明天读取。

## 移交 ( 准备 HANDOFF )
对每个保留发现，输出任务行：
worktree=fix/<slug>
goal=<stop-condition>

## 停止 ( 你为自己保留的边界 )
绝不合并，绝不删除。不够有把握的进入 ./inbox/，交给人工处理。
```

这个技能的六个标题中，有五个对应五个动作；第六个“停止”则是构建者写入循环无法自行推断之边界的地方。循环会忠实执行技能写出的全部内容，也不会补上任何遗漏，因此“停止”部分不是样板话——它是工程师将“控制权保留到哪里”的意图永久写下的唯一位置。漏掉它，循环就会带着并不配得上的自信去合并。

XIV. 附录 B：术语表

表 VII 本文使用的术语

术语	含义
循环	无需人在内循环中即可发现、执行、验证、持久化并重新调度工作的系统。
运行框架	武装单次智能体运行的工具包：工具、允许动作、恢复、“完成”。
动作	循环单次迭代中的五个步骤之一。
部分	实现这些动作的六个组件之一。
生成器	负责写作/编码的智能体。
评估器	单独负责判断的智能体，默认怀疑并通过行动验证。
工作树	Git 机制，为每个并行智能体提供独立工作目录。
技能	写入 SKILL.md 的持久项目知识。
连接器	把循环连接到外部系统的 MCP 接口。
记忆	磁盘上的持久状态，跨单次对话保存。
意图债	反复解释项目的成本，可由技能偿还。
验证债	在“能跑”和“正确”之间累积的未验证输出。

XV. 综合：这份作战手册归结为何

纵览九个章节，全文论证只有一条主线。循环工程是一个分层体系中的第四层；这一体系始于提示，经过上下文与运行框架逐层上升，而它与下面三层的区别在于：它把人从“亲自做事”的位置上移开。循环的单次轮转包含五个动作——发现、移交、验证、持久化、调度——由六个部分实现；循环的失败，本质上就是这些动作被跳过。其中最难的是验证，因为智能体给自己的工作打分时总会夸奖自己，而可靠的补救是结构性的：使用独立的评估器，默认怀疑，以行动代替阅读，并由一个全新的模型依据明确的停止条件来评判它。循环已经在实践中运行，从一位工程师的早晨，到一家企业每周合并上千个机器编写的 pull request；真正让它们可靠的，是约束的质量，而不是模型的规模。它们还会累积四种无声的债：验证债、理解腐蚀、认知让渡和 Token 爆表；这些债彼此强化，并会同时到期。而且，因为循环会放大构建者带入其中的一切，同样的循环由两个人构建，结果也可能完全相反，差别只在一两个检查点，而正是这些检查点决定了后来究竟由谁掌控。真正该带走的那句话，是这个领域在一周内达成共识的那句：不要再只给智能体写提示，而要设计那个给它写提示的系统——但要以打算继续做工程师的人来设计它，而不只是那个按下开始按钮的人。所有一切 本札记 中的技术内容都服务于这一立场。评估器、状态文件、预算上限、敞开的门：每一项都是让人类仍能对一个高速说“是”的机器说“否”的方式。循环之所以是这一代软件实践中最强大的工具，恰恰因为它最忠实地放大其构建者；而忠实的放大器，价值或危险，都取决于输入其中的判断力。

A. 首月实录

对于采用循环的团队，有几条实践观察反复出现，值得直说。第一，能存活下来的循环，是那个靠赢得信任站住脚的小循环，而不是那个一开始就索要信任的宏大循环；应从单个问题的端到端处理起步，只有在检查机制真正抓到错误之后再扩大。第二，工程投入应集中在评估器上——强生成器配弱裁判，只会产出自信的垃圾；一般的生成器配敏锐的裁判，虽慢却可靠，而后者才会形成复利。第三，人工审查点不是循环建立信任后就该拆掉的临时脚手架；它是让循环持续可信的永久特性，拆掉它的那一天，理解腐蚀也就真正开始。第四，预算上限应按“迟早会有东西整夜空转”来设，因为这种事终会发生；而上限决定了它只是日志里的一个插曲，还是账单上的一行费用。这些单独看都不意外。真正让团队意外的是：一个“就是能跑”的愉快循环体验，会多快侵蚀掉那些让它起作用的纪律；而这种侵蚀在某个早晨真正显形之前，几乎不可见。归根结底，这份行动手册与其说是在教人构建循环——那部分现在确实已经很容易——不如说是在教人如何继续做那种工程师：在任何一个早晨，仍能回答循环刚刚完成的那件事到底是不是对的。

参考文献

- [1] A. Osmani, “循环工程,” personal blog and Substack, 2026年6月.
- [2] P. Steinberger, 关于设计用于提示 coding agents 的循环的帖子, social media, 2026年6月.
- [3] B. Cherny, 关于编写用于提示 Claude 的循环的公开发言, Anthropic, 2026年6月.
- [4] P. Rajasekaran, “构建长时间运行的智能体式应用：生成器/评估器模式,” Anthropic engineering blog, 2026.
- [5] S. Kaliski, “Stripe’ s Minions: 每周 1,300 个 PR,” How I AI podcast, 2026.
- [6] “Model Context Protocol (MCP) specification,” 开放标准, 2025–2026.
- [7] “Goose: an open-source agent framework,” 项目文档, 2025–2026.
- [8] “Claude Code documentation: /loop, /goal, worktrees, skills, automa-tions,” Anthropic, 2026.
- [9] HuaShu, Loop Engineering: Stop Asking Me What It Is, Orange Books, v260615, 2026年6月.

致谢。本文是一篇独立的会议风格综述，建立在 HuaShu 的开放指南 Loop Engineering: Stop Asking Me What It Is (Orange Books, v260615, 2026年6月) 的框架之上。其框架与引文表述归功于 Addy Osmani；生成器/评估器相关发现归功于 Prithvi Rajasekaran (Anthropic)；企业案例归功于 Steve Kaliski (Stripe)。所有产品细节均可能变化；请以各工具官方文档为准。